

STensor/CovariantStuctures

Setup

```
In[*]:= << mGRG`STensor`

In[*]:= $PreRead = ReplaceAll[#,
    expr_String => StringReplace[expr, "SP" -> "mGRG`STensor`Private"]] &;

In[*]:= DefKind[Greek, Alphabet["Greek"]];
DefKind[Capital, ToUpperCase /@ Alphabet[]];
DefKind[Bar, "b" <> # & /@ Alphabet[]];
DefKind[Dot, "d" <> # & /@ Alphabet[]];
DefKind[Hat, "h" <> # & /@ Alphabet[]];

In[*]:= Tdefine[f[]]; Tdefine[v[ua], 1];
Tdefine[T, 2]; Tdefine[F, "-ba"]

In[*]:= Tdefine[CF[uA, uB], "-ba"]

In[*]:= SetAttributes[{c1, c2}, Constant]
```

Define Derivative Operators and Metrics

DerivativeOperatorQ

```
In[*]:= DerivativeOperatorQ::usage
Out[*]:= DerivativeOperatorQ[opName] returns True if opName
is a defined derivative operator, and False otherwise.
```

DefDerivativeOperator로 정의된 미분 연산자인지 묻는다:

```
In[*]:= DerivativeOperatorQ /@ {CD, BD, LD}
Out[*]:= {True, False, False}
```

DefDerivativeOperator and UndefDerivativeOperator

In[*]:= **DefDerivativeOperator::usage**

Out[*]=

DefDerivativeOperator[covD, prtStr, kind, opts] defines a new derivative operator 'covD'. The print string 'prtStr' and the Kind 'kind' are optional, defaulting to the operator's name and DefaultKind, respectively. Options like TorsionFreeQ can be specified.

CD type의 미분 연산자를 정의한다. 인자는 연산자 이름, 옵션으로 출력을 위한 문자열, 그리고 연산자가 속한 Kind, TorsionFreeQ가 있다.

In[*]:= **DefDerivativeOperator[CovD, "D"]**

CovD[la, T[lb, lc]]

Out[*]=

$D_a T_{bc}$

정의된 연산자 CovD와 관련된 IndexedObject들이 자동적으로 정의된다:

In[*]:= **SP`getDerOp[#][CovD] & /@ {Gamma, Riemann, Ricci}**

{GammaCovD[la, lb, uc], RiemannCovD[la, lb, lc, ud], RicciCovD[la, lb]}

Out[*]=

{GammaCovD, RiemannCovD, RicciCovD}

Out[*]=

$\{\Gamma[D]_{ab}{}^c, R[D]_{abc}{}^d, R[D]_{ab}\}$

연산자가 정의된 Kind에 계량 텐서가 존재하면 스칼라 곡률도 정의된다:

In[*]:= **SP`getDerOp[Scalar][CovD]**

ScalarCovD[]

Out[*]=

ScalarCovD

Out[*]=

R[D]

새로 정의한 미분 연산자가 속한 Kind의 모든 미분 연산자 이름을 알 수 있다. CD는 DefaultKind에 미리 정의된 공변 도함수이다.

In[*]:= **SP`getDerOperators[Latin]**

Out[*]=

{CD, CovD}

리만 텐서, 리치 텐서, 스칼라 곡률 사이의 기본적인 관계가 자동적으로 적용된다:

In[*]:= {RiemannCovD[la, lb, lc, ub], RicciCovD[la, ua]}

Out[*]= $\{R[D]_{ac}, R[D]\}$

선형 미분 연산자이고, 라이프니츠 규칙을 만족한다:

In[*]:= c1 RicciCovD[la, lb] + c2 F[la, lc] × RicciCovD[uc, lb]
CovD[ua, %]

Out[*]= $c1 R[D]_{ab} + c2 F_{ac} R[D]^c_b$

Out[*]= $c1 D^a R[D]_{ab} + c2 D^a R[D]^c_b F_{ac} + c2 D^a F_{ac} R[D]^c_b$

크로네커 델타는 상수 텐서이다:

In[*]:= Kdelta[lb, uc]
CovD[la, %]

Out[*]= δ_b^c

Out[*]= 0

연속된 미분은 연속해서 입력한다:

In[*]:= CovD[la, ub, RicciCovD[lc, ud]]

Out[*]= $D_a D^b R[D]^d_c$

In[*]:= KindOf::usage

Out[*]= KindOf[obj, pos] returns the kind of the object 'obj' at a given index position 'pos'. For an indexed object expression like T[a,b], KindOf[T[a,b], a] returns the kind of index 'a'.

연산자가 정의된 Kind를 알 수 있다:

In[*]:= KindOf[CovD]

Out[*]= Latin

In[*]:= MetricSpaceQ::usage

Out[*]= MetricSpaceQ[kind] returns True if the specified 'kind' is a metric space, and False otherwise.

```
In[*]:= CoordinateBasisQ::usage
```

```
Out[*]=
```

```
CoordinateBasisQ[kind] returns True if the
basis associated with 'kind' is a coordinate basis.
```

MetricSpaceQ와 CoordinateBasisQ는 Kind에 속한 성질이다. 기본적으로 정의된 Latin Kind에는 계량 텐서 Metricg가 있고, 좌표 기준(Coordinate Basis)으로 설정되었다:

```
In[*]:= MetricSpaceQ /@ {Latin, Capital}
```

```
CoordinateBasisQ /@ {Latin, Capital}
```

```
Out[*]=
```

```
{True, False}
```

```
Out[*]=
```

```
{True, False}
```

Capital Kind의 미분 연산자를 정의하자:

```
In[*]:= DefDerivativeOperator[CCovD, "D", Capital]
```

```
SP`getDerOp[#][CCovD] & /@ {Gamma, Ricci, Riemann}
```

```
Out[*]=
```

```
{GammaCCovD, RicciCCovD, RiemannCCovD}
```

Capital Kind에는 계량 텐서가 정의되지 않았으므로 스칼라 곡률이 정의되지 않았다:

```
In[*]:= SP`getDerOp[Scalar][CCovD]
```

```
Out[*]=
```

```
mGRG`STensor`Private`getDerOp[Scalar][CCovD]
```

계량 텐서가 없으므로 리만 텐서의 대칭도 첫 번째와 두 번째 인덱스 교환에 대한 반대칭 뿐이다:

```
In[*]:= GetSymmetry[RiemannCCovD] // GStoString
```

```
Out[*]=
```

```
-ba
```

```
In[*]:= TorsionFreeQ::usage
```

```
Out[*]=
```

```
TorsionFreeQ[covD] is a property that is True if the
derivative 'covD' is torsion-free, and False otherwise.
It can be set as an option in DefDerivativeOperator.
```

미분 연산자를 정의할 때 DefDerivativeOperator는 기본적으로는 TorsionFree를 가정한다. 이러한 기본 동작을 변경하려면 옵션으로 조정한다:

```
In[*]:= DefDerivativeOperator[CovD, "D", TorsionFreeQ → False]
TorsionFreeQ[CovD]
```

```
Out[*]=
False
```

Torsion이 있는 경우 Affine Connection과 리치 텐서는 아무런 대칭도 없다:

```
In[*]:= GetSymmetry /@ {GammaCovD, RicciCovD}
```

```
Out[*]=
{GenSet[], GenSet[]}
```

Greek Kind의 미분 연산자를 정의하자:

```
In[*]:= DefDerivativeOperator[GCovD, "D", Greek]
```

TorsionFree에 대한 옵션이 없으면 기본적으로 TorsionFree 미분 연산자이다:

```
In[*]:= TorsionFreeQ[GCovD]
```

```
Out[*]=
True
```

비록 TorsionFree 이지만 좌표 기준(Coordinate Basis)으로 설정되지 않았으므로 Affine Connection의 대칭은 없다. (참고: 좌표 기준으로 설정하려면 On[CoordinateBasisFlag[Greek]] 명령을 사용한다.)

```
In[*]:= CoordinateBasisQ[Greek]
GetSymmetry[GammaGCovD]
```

```
Out[*]=
False
```

```
Out[*]=
GenSet[]
```

정의된 미분 연산자를 프로그램에서 제거할 수 있다:

```
In[*]:= UndefDerivativeOperator[CovD]
```

```
In[*]:= DerivativeOperatorQ[CovD]
GammaCovD[la, lb, lc]
```

```
Out[*]=
False
```

```
Out[*]=
GammaCovD[la, lb, lc]
```

나머지 연산자도 정의를 취소하자:

```
In[*]:= UndefDerivativeOperator /@ {CCovD, GCovD};
```

프로그램에서 기본적인 공변 도함수인 CD는 제거하지 못한다:

```
In[*]:= UndefDerivativeOperator[CD]
```

Msg: Reserved name CD cannot be removed !

```
Out[*]:=
```

```
$Failed
```

프로그램이 사용할 것으로 예약된 이름 리스트이다:

```
In[*]:= SP`reservedNameList
```

```
Out[*]:=
```

```
{i, All, C, CD, CoXD, D, e, Epsilon, ErrorT, HeadQs,
 HodgeStar, IndexQs, IP, Kdelta, Latin, LD, N, O, Structuref,
 Torsion, Tscalar, XD, XP, VD, Metricg, CovDs, Gamma, Ricci,
 Riemann, Scalar, GammaCD, RicciCD, RiemannCD, ScalarCD, BD}
```

MetricQ

```
In[*]:= MetricQ::usage
```

```
Out[*]:=
```

```
MetricQ[metric] returns True if 'metric'
 is a defined metric tensor, and False otherwise.
```

DefMetric으로 정의한 것인지 묻는다:

```
In[*]:= MetricQ /@ {Metricg, Phi}
```

```
Out[*]:=
```

```
{True, False}
```

DefMetric and UndefMetric

```
In[*]:= DefMetric::usage
```

```
Out[*]:=
```

```
DefMetric[metric, prtStr, kind] defines a
 metric tensor. The optional arguments are 'prtStr' for
 the print string and 'kind' for the kind of the metric.
```

```
In[*]:= UndefMetric::usage
```

```
Out[*]:=
```

```
UndefMetric[metric] removes the definition of the
 metric tensor 'metric' and all associated properties. It also
 updates related derivative operators and curvature tensors.
```

계량 텐서를 정의한다. 인자는 계량 텐서의 이름, 옵션으로 출력을 위한 문자열, 계량 텐서가 속한 Kind가 있다.

DefaultKind에서는 이름이 Metricg인 계량 텐서가 항상 정의되어 있다. 따라서 DefaultKind에서 계량 텐서를 정의하려면 기본 계량 텐서인 Metricg를 제거해야 한다.

기본 계량 텐서를 제거하지 말고 DefaultKind을 Capital로 옮기자:

```
In[*]:= SetDefaultKind[Capital]
```

DefaultKind가 변하면 CD, Metricg, Epsilon도 그 Kind를 따라 간다:

```
In[*]:= KindOf /@ {CD, Metricg, Epsilon}
```

```
Out[*]=
```

```
{Capital, Capital, Capital}
```

```
In[*]:= {GetEpsilon[Latin], GetEpsilon[Capital]}
```

```
Out[*]=
```

```
{EpsilonLatin, Epsilon}
```

이제 Latin Kind에 미분 연산자와 계량 텐서를 도입하자:

```
In[*]:= DefDerivativeOperator[CovD, "D", Latin]
DefMetric[Phi, "Φ", Latin]
```

```
In[*]:= MetricSpaceQ /@ {Latin, Capital}
MetricQ /@ {Phi, Metricg}
```

```
Out[*]=
```

```
{True, True}
```

```
Out[*]=
```

```
{True, True}
```

계량 텐서의 인덱스가 '위-아래' 또는 '아래-위' 첨자이면서 적합한 Kind이면 자동적으로 Kdelta로 변환다:

```
In[*]:= {Phi[la, lb], Phi[la, ub], Phi[lA, uB]}
{Metricg[lA, lB], Metricg[uA, lB], Metricg[la, ub]}
```

```
Out[*]=
```

```
{Φab, δab, ΦAB}
```

```
Out[*]=
```

```
{gAB, δBA, gab}
```

Kdelta의 인덱스 순서는 의미가 없다: $\delta_a^b \equiv +\delta^b_a$.

Kdelta는 상수 텐서이다:

```
In[*]:= {Phi[la, lb], Phi[la, ub], Phi[-1, 2]}
CovD[lc, #] & /@ %
```

```
Out[*]=
```

```
{Φab, δab, Φ12}
```

```
Out[*]=
```

```
{DcΦab, 0, DcΦ12}
```

Metricg는 CD의 공변 상수이다:

```
In[*]:= {Metricg[lA, lB], Metricg[lA, uB], Metricg[-1, 2]}
CD[lC, #] & /@ %
```

```
Out[*]=
```

$$\{g_{AB}, \delta_A^B, g_1^2\}$$

```
Out[*]=
```

$$\{0, 0, \nabla_C g_1^2\}$$

정의한 계량 텐서를 취소한다:

```
In[*]:= UndefMetric[Phi]
MetricQ[Phi]
```

```
Out[*]=
```

```
False
```

SetMetricCompatible and ClearMetricCompatible

```
In[*]:= SetMetricCompatible::usage
```

```
Out[*]=
```

SetMetricCompatible[covD, metric] declares that the covariant derivative 'covD' is compatible with the 'metric', meaning the covariant derivative of the metric with respect to 'covD' is zero. This also implies the volume form is covariantly constant.

```
In[*]:= ClearMetricCompatible::usage
```

```
Out[*]=
```

ClearMetricCompatible[covD, metric] removes the metric-compatibility property between the covariant derivative 'covD' and the 'metric'.

```
In[*]:= DefMetric[Phi, "Φ", Latin]
```

공변 도함수에 대해 공변 상수가 되는 계량 텐서를 설정한다:

```
In[*]:= SetMetricCompatible[CovD, Phi]
```

계량 텐서 Phi가 CovD에 대해 공변 상수임을 확인한다:

```
In[*]:= {Phi[la, lb], Phi[la, ub], Phi[-1, 2]}
CovD[lc, #] & /@ %
```

```
Out[*]=
```

$$\{\Phi_{ab}, \delta_a^b, \Phi_1^2\}$$

```
Out[*]=
```

$$\{0, 0, D_C \Phi_1^2\}$$


```
In[*]:= {CovD[la, Phi[lb, uc]], CovD[-1, Phi[lb, uc]], CovD[la, Phi[-1, 2]],
          CovD[-1, Phi[-1, 2]], CovD[la, Phi[-1, -2]], CovD[-1, Phi[-1, -2]]}
```

```
Out[*]=
```

```
{0, 0, Da⊕12, D1⊕12, Da⊕12, D1⊕12}
```

계량 텐서 뿐만 아니라 Epsilon 텐서도 공변 상수이다. (정확히는 Epsilon 텐서가 속한 Kind에 차원이 설정되었고, 인덱스의 갯수가 그 차원과 일치한 경우에만 공변 상수이다):

```
In[*]:= CovD[la, #] & /@ {Epsilon[lA, lB], EpsilonLatin[lb, lc, ld]}
          CD[lA, #] & /@ {Epsilon[lB, lC], EpsilonLatin[lb, lc, ld]}
```

```
Out[*]=
```

```
{Da∈AB, Da∈[Latin]bcd}
```

```
Out[*]=
```

```
{∇A∈BC, ∇A∈[Latin]bcd}
```

```
In[*]:= SetDimension[3, Latin]
```

```
In[*]:= CovD[la, #] & /@ {Epsilon[lb, lc], EpsilonLatin[lb, lc, ld]}
```

```
Out[*]=
```

```
{Da∈bc, 0}
```

```
In[*]:= ClearDimension[Latin]
```

```
In[*]:= CovD[la, #] & /@ {Epsilon[lb, lc], EpsilonLatin[lb, lc, ld]}
```

```
Out[*]=
```

```
{Da∈bc, Da∈[Latin]bcd}
```

공변 도함수와 공변 상수 사이의 설정을 취소한다:

```
In[*]:= ClearMetricCompatible[CovD, Phi]
          {CovD[la, Phi[lb, lc]], CovD[la, EpsilonLatin[lb, lc, ld]]}
```

```
Out[*]=
```

```
{Da⊕bc, Da∈[Latin]bcd}
```

정의한 미분 연산자를 취소한다:

```
In[*]:= UndefDerivativeOperator[CovD]
          DerivativeOperatorQ[CovD]
```

```
Out[*]=
```

```
False
```

정의한 계량 텐서를 취소한다:

```
In[*]:= UndefMetric[Phi];
MetricQ[Phi]
```

```
Out[*]:= False
```

DefaultKind로 복귀한다:

```
In[*]:= SetDefaultKind[Latin]
```

각각의 Kind마다 한 개의 계량 텐서만 허용된다. 그런데 DefaultKind인 경우 (MetricFlag이 On이면) 프로그램에서 이미 Metric이라는 이름으로 기본 계량 텐서가 정의되어 있다. 따라서 DefaultKind에 새로운 계량 텐서를 도입하려면 먼저 MetricFlag을 Off 시켜서 Metric를 삭제해야 한다.

```
In[*]:= DefMetric[Eps, "ε"]
```

... **Msg:** Use On [MetricFlag] or Off [MetricFlag] to change the metric state of DefaultKind Latin

```
Out[*]:= $Failed
```

```
In[*]:= Off[MetricFlag]
```

DefaultKind에 다른 계량 텐서 h를 도입한다:

```
In[*]:= DefMetric[h];
MetricQ[h]
SP`getMetricSymmetry[h]
```

```
Out[*]:= True
```

```
Out[*]:= 1
```

계량 텐서와 Kdelta 관계가 성립한다:

```
In[*]:= {h[la, lb], h[la, ub], h[lA, uB]} (* h's kind is Latin. *)
```

```
Out[*]:= {hab, δab, hAB}
```

새로 도입한 계량 텐서에는 공변 도함수가 아직 설정되지 않았다:

```
In[*]:= SP`getCovDs[h]
```

```
Out[*]:= {}
```

h에 대한 공변 도함수를 설정한다:

```
In[*]:= SetMetricCompatible[CD, h]
        SP`getCovDs[h]
```

```
Out[*]= {CD}
```

새로 도입한 계량 텐서를 취소한다:

```
In[*]:= UndefMetric[h]
        MetricQ[h]
```

```
Out[*]= False
```

DefaultKind에 기본 계량 텐서 Metricg를 복귀시킨다:

```
In[*]:= On[MetricgFlag]
        GetMetric[Latin]
```

```
Out[*]= Metricg
```

```
In[*]:= {GetEpsilon[Latin], MetricQ[Metricg], SP`getCovDs[Metricg]}
```

```
Out[*]= {Epsilon, True, {CD}}
```

MetricSpaceQ, GetMetric, and GetEpsilon

```
In[*]:= MetricSpaceQ::usage
```

```
Out[*]= MetricSpaceQ[kind] returns True if the
        specified 'kind' is a metric space, and False otherwise.
```

```
In[*]:= GetMetric::usage
```

```
Out[*]= GetMetric[kind] returns the unique
        metric tensor associated with the specified 'kind'.
```

```
In[*]:= GetEpsilon::usage
```

```
Out[*]= GetEpsilon[kind] returns the Levi-Civita
        tensor (volume form) associated with the specified 'kind'.
```

DefKind로 정의한 계열의 계량 텐서 관련 속성을 알 수 있다:

In[*]:= SP`definedKindList

Out[*]= {Latin, Greek, Capital, Bar, Dot, Hat}

In[*]:= MetricSpaceQ /@ SP`definedKindList

Out[*]= {True, False, False, False, False, False}

In[*]:= GetMetric /@ SP`definedKindList

Out[*]= {Metricg, Null, Null, Null, Null, Null}

In[*]:= GetEpsilon /@ SP`definedKindList

Out[*]= {Epsilon, EpsilonGreek, EpsilonCapital, EpsilonBar, EpsilonDot, EpsilonHat}

MetricCompatibleQ

In[*]:= MetricCompatibleQ::usage

Out[*]= MetricCompatibleQ[op, metric] returns True if the operator 'op' is compatible with the 'metric' (i.e., the covariant derivative of the metric with respect to the operator is zero), and False otherwise.

첫 번째 인자인 공변 도함수와 그 공변 도함수가 속한 Kind의 유일한 계량 텐서가 서로 부합하는가를 묻는다:

In[*]:= MetricCompatibleQ /@ {CD, BD, LD}

Out[*]= {True, False, False}

첫 번째 인자와 두 번째 인자가 공변 도함수와 공변 상수 관계 인가를 묻는다:

In[*]:= MetricCompatibleQ[#, Metricg] & /@ {CD, BD, LD}

Out[*]= {True, False, False}

CovDs 옵션으로 임의의 연산자를 (임시로) 공변 도함수로 지정할 수 있다:

In[*]:= MetricCompatibleQ[#, Metricg, {CovDs → {BD, LD}}] & /@ {CD, BD, LD}

Out[*]= {True, True, True}

두 번째 인자인 인덱스의 Kind를 고려한다:

```
In[*]:= MetricCompatibleQ[#, la] & /@ {CD, BD, LD}
SP`covariantNameQ[#, IndexToKind[la], {}] & /@ {CD, BD, LD}
```

```
Out[*]= {True, False, False}
```

```
Out[*]= {True, False, False}
```

LD는 CD type의 연산자가 아니다: (참고: 'KindOf[CD, _]'는 항상 DefaultKind이고 'KindOf[BD, LA]'는 Capital이다.)

```
In[*]:= MetricCompatibleQ[#, {CovDs → {BD, LD}}] & /@ {CD, BD, LD}
```

```
Out[*]= {True, True, True}
```

두 번째 인자가 인덱스이면 MetricCompatibleQ 함수의 첫 번째 인자는 CD type 연산자인 경우만 다룬다:

```
In[*]:= MetricCompatibleQ[#, la, {CovDs → {BD, LD}}] & /@ {CD, BD, LD}
MetricCompatibleQ[#, lA, {CovDs → {BD, LD}}] & /@ {CD, BD, LD}
```

```
Out[*]= {True, True, False}
```

```
Out[*]= {False, True, False}
```

covariantNameQ 함수의 첫 번째 인자는 모든 type의 연산자를 다룬다:

```
In[*]:= SP`covariantNameQ[#, IndexToKind[la], {CovDs → {BD, LD}}] & /@ {CD, BD, LD}
SP`covariantNameQ[#, IndexToKind[lA], {CovDs → {BD, LD}}] & /@ {CD, BD, LD}
```

```
Out[*]= {True, True, True}
```

```
Out[*]= {False, True, True}
```

각 계량 텐서마다 공변 도함수 리스트를 내부적으로 관리하고 있다:

```
In[*]:= SP`getCovDs[Metricg] = {CD, BD}; (* 예를 들기 위한 설정 *)
SP`covariantNameQ[#, Latin, {}] & /@ {CD, BD, LD}
SP`covariantNameQ[#, Capital, {}] & /@ {CD, BD, LD}
```

```
Out[*]= {True, True, False}
```

```
Out[*]= {False, False, False}
```

Metricg의 기본 공변 도함수는 CD이다:

```
In[*]:= SP`getCovDs[Metricg] = {CD}; (* default *)
MetricCompatibleQ[#, Metricg] & /@ {CD, BD, LD}
SP`covariantNameQ[#, Latin, {}] & /@ {CD, BD, LD}
```

```
Out[*]:= {True, False, False}
```

```
Out[*]:= {True, False, False}
```

Pre-defined Tensors

Kdelta

Kdelta는 모든 Kind에서 사용 가능하다.

Kdelta 인덱스는 ‘아래-위’ 또는 ‘위-아래’ 첨자인 경우만 의미가 있다. 그런데 계량 텐서 인덱스가 ‘아래-위’ 또는 ‘위-아래’이면 자동으로 Kdelta로 바뀐다:

```
In[*]:= {Metricg[la, lb], Metricg[la, ub], Metricg[ua, lb], Metricg[ua, ub]}
```

```
Out[*]:= {gab, δab, δab, gab}
```

Kdelta 인덱스가 ‘아래-아래’ 또는 ‘위-위’이면 잘못된 표현으로 간주한다:

```
In[*]:= {Kdelta[la, lb], Kdelta[lA, lB], Kdelta[uA, uB]} // SyntaxCheck
```

```
Msg: Kdelta has wrong -dn/up indices: {la, lb}
```

```
Msg: Kdelta has wrong -dn/up indices: {lA, lB}
```

```
Msg: Kdelta has wrong -dn/up indices: {uA, uB}
```

```
General: Further output of Msg::err will be suppressed during this calculation.
```

```
Out[*]:= {δab, δAB, δAB}
```

Kdelta는 상수 텐서이다:

```
In[*]:= {CD[la, Kdelta[lb, uc]], BD[la, Kdelta[lb, uc]], LD[v, Kdelta[lb, uc]]}
```

```
Out[*]:= {0, 0, 0}
```

Kdelta의 성분 값은 대각선이 아닌 영역에서는 0이고, 대각선 요소의 값은 +1이다:

```
In[*]:= Table[Kdelta[-i, j], {i, 3}, {j, 3}] // MatrixForm
```

```
Out[*]//MatrixForm=
```

$$\begin{pmatrix} 1 & 0 & 0 \\ 0 & 1 & 0 \\ 0 & 0 & 1 \end{pmatrix}$$

```
In[*]:= Table[Kdelta[i, -j], {i, 3}, {j, 3}] // MatrixForm
Out[*]//MatrixForm=
```

$$\begin{pmatrix} 1 & 0 & 0 \\ 0 & 1 & 0 \\ 0 & 0 & 1 \end{pmatrix}$$

Epsilon, Torsion and Structure constants

Epsilon, Torsion, Structuref는 각각의 Kind마다 DefKind 명령으로부터 생성된다:

```
In[*]:= {Epsilon[la, lb], EpsilonGreek[lμ, lv], EpsilonCapital[lA, lB],
          EpsilonBar[lba, lbb], EpsilonDot[l da, l db], EpsilonHat[lha, lhb]}
```

```
Out[*]=
```

$$\{\epsilon_{ab}, \epsilon[\text{Greek}]_{\mu\nu}, \epsilon[\text{Capital}]_{AB}, \epsilon[\text{Bar}]_{\overline{ab}}, \epsilon[\text{Dot}]_{\dot{ab}}, \epsilon[\text{Hat}]_{\hat{ab}}\}$$

```
In[*]:= {Torsion[la, lb, uc], TorsionGreek[lμ, lv, up],
          TorsionCapital[lA, lB, uC], TorsionBar[lba, lbb, ubc],
          TorsionDot[l da, l db, u dc], TorsionHat[lha, lhb, uhc]}
```

```
Out[*]=
```

$$\{t_{ab}{}^c, t[\text{Greek}]_{\mu\nu}{}^\rho, t[\text{Capital}]_{AB}{}^C, t[\text{Bar}]_{\overline{ab}}{}^{\overline{c}}, t[\text{Dot}]_{\dot{ab}}{}^{\dot{c}}, t[\text{Hat}]_{\hat{ab}}{}^{\hat{c}}\}$$

Structuref는 좌표 기준이 아닐 때에만 IndexedObject로 존재한다:

```
In[*]:= Off[CoordinateBasisFlag[Latin]]
          Structuref[la, lb, uc]
```

```
Out[*]=
```

$$f_{ab}{}^c$$

```
In[*]:= On[CoordinateBasisFlag[Latin]] (* 기본으로 복귀 *)
          Structuref[la, lb, uc]
```

```
Out[*]=
```

$$\text{Structuref}[la, lb, uc]$$

Epsilon and Metricg

```
In[*]:= {Metricg[la, lb], Epsilon[la, lb, lc, ld]}
```

```
Out[*]=
```

$$\{g_{ab}, \epsilon_{abcd}\}$$

Epsilon 텐서는 rank가 DefaultKind의 차원과 동일한 완전 반대칭 텐서이다. DefaultKind의 차원이 설정되지 않았으면 임의의 rank를 갖는다:

```
In[*]:= {Epsilon[la], Epsilon[lb, la],
          Epsilon[la, lc, lb], Epsilon[lb, la, ld, lc]} // SyntaxCheck
```

```
Out[*]=
```

$$\{\epsilon_a, \epsilon_{ba}, \epsilon_{acb}, \epsilon_{badc}\}$$

DefaultKind의 차원을 설정한 경우:

```
In[*]:= SetDimension[3];
{Epsilon[la], Epsilon[lb, la],
 Epsilon[la, lc, lb], Epsilon[lb, la, ld, lc]} // SyntaxCheck
```

Msg: invalid number of indices {la} for Epsilon

Msg: invalid number of indices {lb, la} for Epsilon

Msg: invalid number of indices {lb, la, ld, lc} for Epsilon

General: Further output of Msg::err will be suppressed during this calculation.

```
Out[*]=
```

$$\{\epsilon_a, \epsilon_{ba}, \epsilon_{acb}, \epsilon_{badc}\}$$

설정된 차원을 취소한다:

```
In[*]:= ClearDimension[]
```

계량 텐서의 텐서 인덱스가 '아래-위' 또는 '위-아래' 첨자이면 Kdelta로 바뀐다:

```
In[*]:= {Metricg[la, lb], Metricg[ua, lb], Metricg[-1, 2]}
```

```
Out[*]=
```

$$\{g_{ab}, \delta^a_b, g_1^2\}$$

Metricg와 Epsilon은 CD에 대해 공변 상수이다:

```
In[*]:= SetDimension[4]
```

```
In[*]:= CD[la, #] & /@ {Metricg[lb, lc], Epsilon[lb, lc, ld], Epsilon[lb, lc, ld, le]}
```

```
Out[*]=
```

$$\{0, \nabla_a \epsilon_{bcd}, 0\}$$

```
In[*]:= ClearDimension[]
```

```
In[*]:= CD[la, #] & /@ {Metricg[lb, lc], Epsilon[lb, lc, ld], Epsilon[lb, lc, ld, le]}
```

```
Out[*]=
```

$$\{0, \nabla_a \epsilon_{bcd}, \nabla_a \epsilon_{bcde}\}$$

CD-related Tensors

CD는 항상 DefaultKind에 속하도록 기본적으로 정의된 공변 도함수이다. DefDerivativeOperator로 정의하는 다른 CD type의 미분 연산자와 동일하게 다음과 같은 텐서들이 자동으로 정의된다:


```
In[*]:= {GammaCD[la, lb, uc], RicciCD[la, lb], RiemannCD[la, lb, lc, ud], ScalarCD[]}
```

```
Out[*]= { $\Gamma_{ab}^c$ ,  $R_{ab}$ ,  $R_{abc}^d$ ,  $R$ }
```

SetDefaultKind

```
In[*]:= SetDefaultKind::usage
```

```
Out[*]= SetDefaultKind[kind] sets the default kind for all subsequent operations.  
This affects which indices, metrics, and derivatives are used by default.
```

Latin

DefaultKind의 기본은 Latin으로 설정되어 있다:

```
In[*]:= DefaultKind
```

```
Out[*]= Latin
```

```
In[*]:= SP`getDerOperators[DefaultKind]
```

```
Out[*]= {CD}
```

```
In[*]:= {MetricSpaceQ[DefaultKind], GetMetric[DefaultKind]}
```

```
Out[*]= {True, Metricg}
```

```
In[*]:= {GetEpsilon[Latin], MetricQ[Metricg], SP`getCovDs[Metricg]}
```

```
Out[*]= {Epsilon, True, {CD}}
```

```
In[*]:= {Metricg[ua, lb], Metricg[lA, uB]}
```

```
Out[*]= { $\delta_b^a$ ,  $g_A^B$ }
```

```
In[*]:= CD[lc, #] & /@ {Metricg[la, lb], Metricg[lA, lB]}
```

```
Out[*]= {0,  $\nabla_C g_{AB}$ }
```

In[*]:= **CD**[lc, #] & /@ {Epsilon[la, lb], Epsilon[lA, lB]} (* Dimension 설정이 안된 경우 *)

Out[*]=
 $\{\nabla_c \in_{ab}, \nabla_c \in_{AB}\}$

In[*]:= **GetDimension**[Latin]

Out[*]=
GetDimension[Latin]

In[*]:= **SetDimension**[3]

In[*]:= **CD**[lf, #] & /@ {Epsilon[la, lb], Epsilon[la, lb, lc], Epsilon[la, lb, lc, ld]}

Out[*]=
 $\{\nabla_f \in_{ab}, 0, \nabla_f \in_{abcd}\}$

In[*]:= **ClearDimension**[]

In[*]:= **CD**[lf, #] & /@ {Epsilon[la, lb], Epsilon[la, lb, lc], Epsilon[la, lb, lc, ld]}

Out[*]=
 $\{\nabla_f \in_{ab}, \nabla_f \in_{abc}, \nabla_f \in_{abcd}\}$

Spacetime

DefaultKind를 변경하여도 공변 연산자 CD와 계량 텐서 Metricg, Epsilon 텐서는 항상 변경된 Default-Kind에 속하도록 설정된다.

DefaultKind를 Greek로 변경한다:

In[*]:= **SetDefaultKind**[Greek]
DefaultKind

Out[*]=
Greek

In[*]:= {SP`getDerOperators[DefaultKind],
GetMetric[DefaultKind], **GetEpsilon**[DefaultKind]}

Out[*]=
{{CD}, Metricg, Epsilon}

In[*]:= **KindOf** /@ {CD, Metricg, Epsilon}

Out[*]=
{Greek, Greek, Greek}

In[*]:= **Metricg**[la, ub], **Metricg**[uμ, lv]]

Out[*]=

$\{g_a^b, \delta_{\nu}^{\mu}\}$

In[*]:= **CD**[lμ, **Metricg**[lb, lc]], **CD**[lρ, **Metricg**[lμ, lv]]]

Out[*]=

$\{\nabla_{\mu} g_{bc}, 0\}$

In[*]:= **SetDimension**[2] (* 임시 설정 *)

In[*]:= **CD**[la, **Epsilon**[lb, lc]], **CD**[lμ, **Epsilon**[lb, lc]],
CD[la, **Epsilon**[lμ, lv]], **CD**[lρ, **Epsilon**[lμ, lv]]]

Out[*]=

$\{\nabla_a \epsilon_{bc}, \nabla_{\mu} \epsilon_{bc}, \nabla_a \epsilon_{\mu\nu}, 0\}$

In[*]:= **ClearDimension**[]

In[*]:= **CD**[la, **Epsilon**[lb, lc]], **CD**[lμ, **Epsilon**[lb, lc]],
CD[la, **Epsilon**[lμ, lv]], **CD**[lρ, **Epsilon**[lμ, lv]]]

Out[*]=

$\{\nabla_a \epsilon_{bc}, \nabla_{\mu} \epsilon_{bc}, \nabla_a \epsilon_{\mu\nu}, \nabla_{\rho} \epsilon_{\mu\nu}\}$

Default

기본 설정으로 복귀한다:

In[*]:= **SetDefaultKind**[Latin]
DefaultKind

Out[*]=

Latin

In[*]:= **SP`getDerOperators**[DefaultKind],
GetMetric[DefaultKind], **GetEpsilon**[DefaultKind]]

Out[*]=

$\{\{CD\}, Metricg, Epsilon\}$

In[*]:= **SetDimension**[2] (* 임시 설정 *)

In[*]:= **CD**[la, **Epsilon**[lb, lc]], **CD**[lμ, **Epsilon**[lb, lc]],
CD[la, **Epsilon**[lμ, lv]], **CD**[lρ, **Epsilon**[lμ, lv]]]

Out[*]=

$\{0, \nabla_{\mu} \epsilon_{bc}, \nabla_a \epsilon_{\mu\nu}, \nabla_{\rho} \epsilon_{\mu\nu}\}$

In[*]:=	ClearDimension[]
In[*]:=	{CD[la, Epsilon[lb, lc]], CD[lμ, Epsilon[lb, lc]], CD[la, Epsilon[lμ, lv]], CD[lρ, Epsilon[lμ, lv]]}
Out[*]=	{$\nabla_{\mathbf{a} \in \mathbf{bc}}$, $\nabla_{\mu \in \mathbf{bc}}$, $\nabla_{\mathbf{a} \in \mu \nu}$, $\nabla_{\rho \in \mu \nu}$}

KindOf

In[*]:=	SetDefaultKind[Capital]
In[*]:=	DefTensor[H[la, lμ]]

Kdelta

Kdelta는 모든 Kind에서 사용한다. 따라서 Kdelta의 Kind는 사용한 인덱스 Kind로부터 결정한다:

In[*]:=	{KindOf[Kdelta[la, ub], la], KindOf[Kdelta[la, ub], ub], KindOf[Kdelta[la, ub], lc]}
Out[*]=	{Latin, Latin, All}
In[*]:=	{KindOf[Kdelta], KindOf[Kdelta, la], KindOf[Kdelta, lA]}
Out[*]=	{All, Latin, Capital}
In[*]:=	{KindOf[Kdelta], KindOf[Kdelta, 2], KindOf[Kdelta, 3]}
Out[*]=	{All, Capital, Capital}

참고: 성분 인덱스의 Kind는 DefaultKind이고, indexKind 명령의 옵션으로 두 번째 인자가 있으면 그 인자로부터 인덱스 계열을 유추한다:

In[*]:=	IndexToKind[1] IndexToKind[1, All]
Out[*]=	Capital
Out[*]=	All

Epsilon and Metricg

Epsilon, Metricg는 항상 DefaultKind에 속한다:

```
In[*]:= {KindOf[Metricg[la, lb], la],
        KindOf[Metricg[la, lb], lb], KindOf[Metricg[la, lb], lc]}
```

```
Out[*]= {Capital, Capital, Capital}
```

```
In[*]:= {KindOf[Metricg], KindOf[Metricg, 2], KindOf[Metricg, 3]}
```

```
Out[*]= {Capital, Capital, Capital}
```

```
In[*]:= {KindOf[Epsilon[la, lb], la],
        KindOf[Epsilon[la, lb], lb], KindOf[Epsilon[la, lb], lc]}
```

```
Out[*]= {Capital, Capital, Capital}
```

```
In[*]:= {KindOf[Epsilon], KindOf[Epsilon, 2], KindOf[Epsilon, 3]}
```

```
Out[*]= {Capital, Capital, Capital}
```

Operand

그 외의 텐서는 DefTensor 함수로 정의한 바대로 Kind를 유추한다:

```
In[*]:= {KindOf[H[la, lμ], la], KindOf[H[la, lμ], lμ], KindOf[H[la, lμ], lc]}
```

```
Out[*]= {Latin, Greek, Latin}
```

KindOf 함수의 두 번째 인자가 숫자이면 그 숫자에 해당하는 인덱스의 Kind가 바로 텐서의 Kind이다.

```
In[*]:= {KindOf[H], KindOf[H, 2]}
```

```
Out[*]= {Latin, Greek}
```

그런데 두 번째 인자가 없는 경우는 두 번째 인자의 값이 1인 것과 동일하고, 두 번째 인자가 rank의 크기를 벗어나는 정수이면 마지막 번째의 인덱스 Kind가 텐서의 Kind이다:

```
In[*]:= {KindOf[H], KindOf[H, 3]}
```

```
Out[*]= {Latin, Greek}
```

BD

BD 연산자는 Kdelta와 동일하게 모든 Kind에서 사용한다. 따라서 BD의 Kind는 사용한 인덱스 Kind로 결정한다:

```
In[*]:= {KindOf[BD], KindOf[BD, la], KindOf[BD, lμ]}
```

```
Out[*]=
```

```
{All, Latin, Greek}
```

```
In[*]:= {KindOf[BD], KindOf[BD, 1], KindOf[BD, 2]}
```

```
Out[*]=
```

```
{All, Capital, Capital}
```

CD

CD 연산자는 항상 DefaultKind에 속한다:

```
In[*]:= {KindOf[CD], KindOf[CD, la], KindOf[CD, lμ], KindOf[CD, 1], KindOf[CD, 2]}
```

```
Out[*]=
```

```
{Capital, NonKind, NonKind, Capital, Capital}
```

Operator

LD type이 아닌 연산자의 Kind는 기본적으로 defineOperator로 연산자를 정의할 때 설정한 Kind이다:

```
In[*]:= SP`defineOperator[CovD, "D", CD]
```

```
{KindOf[CovD, la], KindOf[CovD, 2]}
```

```
Out[*]=
```

```
{Capital, Capital}
```

KindOf 함수의 첫 번째 인자가 LD type의 '연산자 이름'이면 두 번째 인자의 Kind로 결정한다:

```
In[*]:= Tdefine[ξ[lμ], 1]; SP`defineOperator[LieD, "L", LD, Latin]
```

```
{KindOf[LieD, ξ], KindOf[LieD, v], KindOf[LieD, f]}
```

```
Out[*]=
```

```
{Greek, Latin, Latin}
```

Default

원래의 설정으로 복귀한다:

```
In[*]:= SetDefaultKind[Latin]
```

Pre-defined Tensorial Operators

BD: Ordinary Derivative

In[*]:= **BD::usage**

Out[*]=

BD[aIndex, expr] represents the basis derivative of 'expr' with respect to the basis vector associated with 'aIndex'. For a coordinate basis, this is equivalent to a partial derivative. For a non-coordinate basis, it is defined via the basis matrix. BD[kind][aIndex, expr] can be used to specify the derivative for a particular kind.

CD type의 편미분 연산자이다.

좌표 기준과 비좌표 기준의 출력 표현에 차이가 있다: 좌표 기준에서는 ∂ 로, 비좌표 기준에서는 $\hat{\partial}$ 로 출력된다.

Latin Kind는 프로그램 초기에 좌표 기준으로 설정되었다:

In[*]:= **CoordinateBasisQ[Latin]**
{BD[-1, v[lb]], BD[la, v[lb]]}

Out[*]=

True

Out[*]=

$\{\partial_1 v_b, \partial_a v_b\}$

Capital Kind는 아무런 설정을 하지 않았으므로 기본적으로 비좌표 기준이다:

In[*]:= **CoordinateBasisQ[Capital]**
BD[la, v[lb]]

Out[*]=

False

Out[*]=

$\hat{\partial}_A v_b$

BD는 선형 미분 연산자로 라이프니츠 규칙을 만족한다:

In[*]:= **c1 RicciCD[la, lb] + c2 F[la, lc] × RicciCD[uc, lb]**
BD[ua, %]

Out[*]=

$c1 R_{ab} + c2 F_{ac} R^c_b$

Out[*]=

$c1 \hat{\partial}^a R_{ab} + c2 \hat{\partial}^a R^c_b F_{ac} + c2 \hat{\partial}^a F_{ac} R^c_b$

Kdelta는 상수 텐서이다:

```
In[*]:= {Metricg[lb, uc], Metricg[lb, lc]}
BD[la, #] & /@ %
```

```
Out[*]=
```

$$\{\delta_b^c, g_{bc}\}$$

```
Out[*]=
```

$$\{0, \partial_a g_{bc}\}$$

IndexedObject의 Log나 Power는 ScalarFunction이므로 편미분 연산을 한다:

```
In[*]:= {BD[la, Log[f[]]], BD[la, f[]^2], BD[la, e^f[]], BD[la, x^f[]]}
```

```
Out[*]=
```

$$\left\{ \frac{\partial_a f}{f}, 2 \partial_a f f, e^f \partial_a f, x^{-1+f} \partial_a x f + x^f \partial_a f \log[x] \right\}$$

```
In[*]:= BD[lc, (RicciCD[la, lb] × Metricg[ua, ub])^n] (* 텐서의 Power는 적절한 표현이 아님 *)
```

```
Out[*]=
```

$$n \partial_c R_{ab} g^{ab} (g^{ab} R_{ab})^{-1+n} + n \partial_c g^{ab} R_{ab} (g^{ab} R_{ab})^{-1+n} + \partial_c n \log[g^{\text{Latin\$17772Latin\$17774}} R_{\text{Latin\$17772Latin\$17774}}] (g^{ab} R_{ab})^n$$

스칼라 표현은 Tscalar로 돌려 쓴다:

```
In[*]:= SetAttributes[n, Constant];
BD[lc, Tscalar[RicciCD[la, lb] × RicciCD[ua, ub]]^n]
```

```
Out[*]=
```

$$n \partial_c (R_{ab} R^{ab}) (R_{ab} R^{ab})^{-1+n}$$

비좌표 기준으로 설정하자:

```
In[*]:= Off[CoordinateBasisFlag]
```

연속한 미분은 인자를 연속해서 쓰면 된다:

```
In[*]:= {F[ua, ub], RicciCD[ua, ub]}
BD[lb, la, #] & /@ % (* 인덱스를 자동으로 재조정함 *)
```

```
Out[*]=
```

$$\{F^{ab}, R^{ab}\}$$

```
Out[*]=
```

$$\{\hat{\partial}_a \hat{\partial}_b F^{ba}, \hat{\partial}_a \hat{\partial}_b R^{ba}\}$$

좌표 기준으로 설정하자:

```
In[*]:= On[CoordinateBasisFlag]
```



```
In[*]:= {F[ua, ub], RicciCD[ua, ub]}
BD[lb, la, #] & /@%
```

```
Out[*]= {Fab, Rab}
```

```
Out[*]= {∂a∂bFba, ∂a∂bRba}
```

SyntaxCheckFlag이 Off이면 오류 메시지가 발생하지 않는다:

```
In[*]:= BD[aa, RiemannCD[la, ub, ua, ud]] (* no SyntaxCheck *)
```

```
Out[*]= ∂aaRbd
```

```
In[*]:= BD[aa, RiemannCD[la, ub, ua, ud]] // SyntaxCheck
```

General: aa is not a valid index.

```
Out[*]= ∂aaRbd
```

BD는 모든 종류의 Kind에서 사용된다:

```
In[*]:= {BD[lb, la, CF[ua, ub]], BD[lb, la, F[ua, ub]],
BD[-1, la, F[ua, 1]], BD[ub, la, F[ua, lb]]} // SyntaxCheck
```

```
Out[*]= {∂B∂ACFBA, ∂a∂bFba, ∂1∂aFa1, ∂a∂bFba}
```

```
In[*]:= {BD[lb, la, F[la, lb]], BD[lμ, lν, F[la, lb]],
BD[-1, la, F[la, -1]]} // SyntaxCheck
```

```
Out[*]= {∂B∂AFab, ∂μ∂νFab, ∂1∂AFa1}
```

DefaultKind가 아닌 경우의 성분을 BD의 인자로 사용하려면 명시적으로 그 Kind를 표시해야 한다:

```
In[*]:= BD[Capital][-1, CF[ua, ub]]
```

```
Out[*]= ∂[Capital]1CFAB
```

CD: Covariant Derivative

CD type의 미분 연산자이다:

```
In[*]:= {CD[la, v[ua]], CD[1, v[la]], CD[-1, v[1]]}
```

```
Out[*]= {∇ava, ∇1va, ∇1v1}
```

Metricg에 대한 공변 도함수이다:

$In[*]:=$ `{Metricg[lb, lc], Epsilon[lb, lc, ld, le], Metricg[-1, lc]}`
`CD[la, #] & /@%`
 $Out[*]=$ $\{g_{bc}, \epsilon_{bcde}, g_{1c}\}$
 $Out[*]=$ $\{0, \nabla_a \epsilon_{bcde}, \nabla_a g_{1c}\}$

Volume-form에 대해서는 시공간에 차원이 설정되었고 그 차원과 인덱스의 갯수가 일치한 경우에만 공변도함수가 역할을 한다:

$In[*]:=$ `CD[la, #] & /@ {Epsilon[lb, lc, ld], Epsilon[lb, lc, ld, le]}`
 $Out[*]=$ $\{\nabla_a \epsilon_{bcd}, \nabla_a \epsilon_{bcde}\}$
 $In[*]:=$ `SetDimension[4]`
`CD[la, #] & /@ {Epsilon[lb, lc, ld], Epsilon[lb, lc, ld, le]}`
`ClearDimension[]`
 $Out[*]=$ $\{\nabla_a \epsilon_{bcd}, 0\}$

선형 미분 연산자로 라이프니츠 규칙을 만족한다:

$In[*]:=$ `c1 RicciCD[la, lb] + c2 F[la, lc] × RicciCD[uc, lb]`
`CD[ua, %]`
 $Out[*]=$ $c1 R_{ab} + c2 F_{ac} R^c_b$
 $Out[*]=$ $c1 \nabla^a R_{ab} + c2 \nabla^a R^c_b F_{ac} + c2 \nabla^a F_{ac} R^c_b$
 $In[*]:=$ `CD[la, RiemannCD[ua, lb, lc, ld] × Metricg[ub, uc]]`
 $Out[*]=$ $\nabla_a R^a_{bcd} g^{bc}$

Kdelta는 상수 텐서이다:

$In[*]:=$ `{Metricg[lb, uc], Metricg[-1, lc]}`
`CD[la, #] & /@%`
 $Out[*]=$ $\{\delta_b^c, g_{1c}\}$
 $Out[*]=$ $\{0, \nabla_a g_{1c}\}$

IndexedObject의 Log나 Power는 ScalarFunction이다:

In[*]:= $\{CD[la, Log[f[]]], CD[la, f[]^2], CD[la, e^{f[]}], CD[la, x^{f[]}]\}$

Out[*]= $\left\{ \frac{\nabla_a f}{f}, 2 \nabla_a f f, e^f \nabla_a f, x^{-1+f} \nabla_a x f + x^f \nabla_a f \log[x] \right\}$

In[*]:= **SetAttributes[n, Constant];**
CD[lc, (RicciCD[la, lb] × Metricg[ua, ub])ⁿ] (* 부적절한 연산 *)

Out[*]= $n \nabla_c R_{ab} g^{ab} (g^{ab} R_{ab})^{-1+n}$

In[*]:= **CD[lc, Tscalar[RicciCD[la, lb] × RicciCD[ua, ub]]ⁿ] (* Tscalar 사용 *)**

Out[*]= $n \nabla_c (R_{ab} R^{ab}) (R_{ab} R^{ab})^{-1+n}$

연속한 미분은 인자를 연속해서 쓰면 된다:

In[*]:= **CD[la, lb, lc, F[ld, le]]**
CD[lb, la, f[]]

Out[*]= $\nabla_a \nabla_b \nabla_c F_{de}$

Out[*]= $\nabla_b \nabla_a f$

CD는 항상 DefaultKind의 공변 연산자이다:

In[*]:= **SetDefaultKind[Capital]**

In[*]:= **{CD[lA, Metricg[lB, lC]], CD[la, Metricg[lB, lC]], CD[lA, Metricg[lμ, lB]]}**

Out[*]= $\{0, \nabla_a g_{BC}, \nabla_A g_{\mu B}\}$

원래의 DefaultKind로 복귀한다:

In[*]:= **SetDefaultKind[Latin]**

LD: Lie Derivative

LD type의 Lie 도함수이다:

```
In[*]:= {LD[v, v[la]], LD[v, v[ua]], LD[v, v[1]]}
LD[v, RiemannCD[la, lb, ub, uc]]
```

```
Out[*]=
```

$$\{\mathcal{L}_v v_a, 0, \mathcal{L}_v v^1\}$$

```
Out[*]=
```

$$-\mathcal{L}_v R_a^c$$

선형 연산자로 라이프니츠 규칙을 만족한다:

```
In[*]:= c1 RicciCD[la, lb] + c2 F[la, lc] × RicciCD[uc, lb]
LD[v, %]
```

```
Out[*]=
```

$$c1 R_{ab} + c2 F_{ac} R_b^c$$

```
Out[*]=
```

$$c1 \mathcal{L}_v R_{ab} + c2 F_{ac} \mathcal{L}_v R_b^c + c2 \mathcal{L}_v F_{ac} R_b^c$$

Kdelta는 상수 텐서이다:

```
In[*]:= {Metricg[lb, uc], Metricg[lb, lc]}
LD[v, #] & /@ %
```

```
Out[*]=
```

$$\{\delta_b^c, g_{bc}\}$$

```
Out[*]=
```

$$\{0, \mathcal{L}_v g_{bc}\}$$

IndexedObject의 Log나 Power는 ScalarFunction0이다:

```
In[*]:= {LD[v, Log[f[]]], LD[v, f[]^2], LD[v, e^f[]], LD[v, x^f[]]}
```

```
Out[*]=
```

$$\left\{ \frac{\mathcal{L}_v f}{f}, 2 f \mathcal{L}_v f, e^f \mathcal{L}_v f, x^{-1+f} f \mathcal{L}_v x + x^f \mathcal{L}_v f \log[x] \right\}$$

```
In[*]:= SetAttributes[n, Constant];
LD[v, (RicciCD[la, lb] × Metricg[ua, ub])^n]
```

```
Out[*]=
```

$$n \mathcal{L}_v R_{ab} g^{ab} (g^{ab} R_{ab})^{-1+n} + n \mathcal{L}_v g^{ab} R_{ab} (g^{ab} R_{ab})^{-1+n}$$

```
In[*]:= LD[v, Tscalar[RicciCD[la, lb] × RicciCD[ua, ub]]^n]
```

```
Out[*]=
```

$$n \mathcal{L}_v (R_{ab} R^{ab}) (R_{ab} R^{ab})^{-1+n}$$

LD의 Kind는 첫 번째 인자인 벡터의 계열로 해석한다:

```
In[*]:= {LD[v, v[la]], LD[v, v[ua]], LD[v, v[ua]]}
Out[*]= {LvvA, LvvA, 0}
```

On/Off Flags for Tensorial Expressions

```
In[*]:= Tdefine[R[la, lb], "ba"];
Tdefine[R4[la], 4];
DefDerivativeOperator[CovD, "∇"]
```

EvaluateBDFlag

성분 입력에 대한 BD 연산을 자동적으로 실행시키려면 On 시키고, 그렇지 않으면 Off 시킨다:

```
In[*]:= Off[EvaluateBDFlag]
BD[-1, fun[x]]
Out[*]= ∂1fun[x]
```

```
In[*]:= On[EvaluateBDFlag]
```

실제로 성분 입력에 대해 BD연산을 실행하려면 SetCoordinates로 좌표계를 설정해야 한다:

```
In[*]:= SetCoordinates[{t, x}]
GetCoordinates[Latin]
Out[*]= {t, x}
```

```
In[*]:= {BD[-1, fun[x]], BD[-2, fun[x]]}
Out[*]= {0, fun'[x]}
```

성분 연산을 DefaultKind가 아닌 다른 Kind에서 실행시키려면 그 Kind 이름을 사용해야 한다:

```
In[*]:= {BD[Capital][-1, fun[x]], BD[Capital][-2, fun[x]]}
Out[*]= {∂[Capital]1fun[x], ∂[Capital]2fun[x]}
```

위에서 성분 입력에 대한 BD 연산이 이루어지지 않은 이유는 DefaultKind에서 동작한 SetCoordiantes 명령이 Capital Kind가 아닌 Latin Kind에 좌표계를 설정했기 때문이다:

```
In[*]:= {BD[Latin][-1, fun[x]], BD[Latin][-2, fun[x]]}
Out[*]= {0, fun'[x]}
```

프로그램의 기본 상태로 복귀한다:

```
In[*]:= ClearCoordinates[]
Off[EvaluateBDFlag]
```

KdeltaFlag

KdeltaFlag이 On인 경우 Kdelta 텐서와의 인덱스 쌍(contracted pair)이 자동적으로 조정된다. 기본적으로는 On이다:

```
In[*]:= Off[KdeltaFlag]
{F[la, lb] × Kdelta[ub, lc], R[la, lb] × Kdelta[lc, ub]}
```

Out[*]=

$$\{F_{ab} \delta_c^b, \delta_c^b R_{ab}\}$$

Kdelta와의 인덱스 쌍을 자동적으로 조정하기 위해 On 시킨다:

```
In[*]:= On[KdeltaFlag]
{F[la, lb] × Kdelta[ub, lc], R[la, lb] × Kdelta[lc, ub]}
```

Out[*]=

$$\{F_{ac}, R_{ac}\}$$

```
In[*]:= c1 R[la, lb] × F[ub, lc] + c2 F[la, lc] + c3 R4[la, lb, lc, ud] × Kdelta[ub, ld]
```

Out[*]=

$$c2 F_{ac} + c1 F_c^b R_{ab} + c3 R4_{abc}^b$$

연산자가 있는 경우 Kdelta와의 인덱스 쌍:

```
In[*]:= Kdelta[ub, ld] × CovD[la, F[lb, lc]]
```

Out[*]=

$$\nabla_a F_{dc}$$

```
In[*]:= Kdelta[ub, ld] × BD[la, F[lb, lc]]
```

Out[*]=

$$\partial_a F_{dc}$$

```
In[*]:= Kdelta[lb, uc] × Kdelta[ua, ld] ×
Kdelta[uf, lg] × BD[lh, CovD[lf, R4[la, ub, lc, ud]]]
```

Out[*]=

$$\partial_h \nabla_g R4_a^b{}_b^a$$

```
In[*]:= Kdelta[ua, lb] × CovD[lc, F[lf, ub]]
```

Out[*]=

$$\nabla_c F_f^a$$

여러 개의 Kdelta 인덱스 쌍:

In[*]:= Kdelta[ua, lg] × Kdelta[ub, lf] × R4[la, lb, lc, ld]
 Out[*]= R^4_{gfcd}

텐서 합에서의 Kdelta 인덱스 쌍:

In[*]:= Kdelta[lc, ub] (R[la, lb] + F[la, lb])
 Out[*]= $F_{ac} + R_{ac}$

텐서의 Log와 Power 표현 내부와 외부에서의 Kdelta 인덱스 쌍:

In[*]:= {Log[f[] × R[la, ub] × Kdelta[ua, lb]],
 R[la, ub] + Kdelta[la, ub] (R[lc, ud] × Kdelta[uc, ld])²}
 Out[*]= $\{\text{Log}[f R_a^a], R_a^b + \delta_a^b R_c^{c^2}\}$

In[*]:= {Log[f[] × R[la, uc]] Kdelta[ua, lb], Log[f[] × R[la, uc] × Kdelta[ua, lb]]}
 (* 잘못된 scalar 표현 *)
 Out[*]= $\{\delta_a^b \text{Log}[f R_a^c], \text{Log}[f R_b^c]\}$

Kdelta의 인덱스 두 개가 모두 동일한 Kind에 있어야 재조정된다:

In[*]:= {F[la, lb] × Kdelta[uB, lc],
 R[la, ub] × Kdelta[ua, lb], R[la, uB] × Kdelta[uA, lB]}
 Out[*]= $\{F_{aA} \delta_c^A, R_a^a, R_a^A\}$

In[*]:= {Kdelta[uB, ld] × CovD[la, F[lB, lc]], Kdelta[ub, ld] × CovD[la, F[lb, lc]]}
 Out[*]= $\{\nabla_a F_{Ac} \delta_d^A, \nabla_a F_{dc}\}$

In[*]:= {CovD[la, R[lb, lc]] × Kdelta[ub, ld],
 CovD[la, R[uB, lc]] × Kdelta[lB, ud], CovD[la, R[ub, lC]] × Kdelta[ld, uC]}
 Out[*]= $\{\nabla_a R_{dc}, \nabla_a R_c^A \delta_A^d, \nabla_a R_A^b \delta_d^A\}$

MetricgFlag

DefaultKind에서 기본 계량 텐서 Metricg를 허용하려면 On 시키고, 그렇지 않으면 Off 시킨다. 기본적으로는 On이다.

```
In[*]:= GetMetric /@ {Latin, Greek, Capital}
```

```
Out[*]:= {Metricg, Null, Null}
```

DefaultKind에서 Metricg를 제거한다:

```
In[*]:= Off[MetricgFlag]
GetMetric /@ {Latin, Greek, Capital}
{Metricg[la, lb], Epsilon[la, lb, lc]}
```

```
Out[*]:= {Null, Null, Null}
```

```
Out[*]:= {Metricg[la, lb],  $\epsilon_{abc}$ }
```

DefaultKind에 기본 계량 텐서 Metricg를 도입한다:

```
In[*]:= On[MetricgFlag]
GetMetric /@ {Latin, Greek, Capital}
{Metricg[la, lb], Epsilon[la, lb, lc]}
```

```
Out[*]:= {Metricg, Null, Null}
```

```
Out[*]:= { $g_{ab}$ ,  $\epsilon_{abc}$ }
```

```
In[*]:= GetSymmetry[Metricg]
```

```
Out[*]:= GenSet[{Cycles[{{1, 2}}], 1}]
```

Absorb and Absorbg

```
In[*]:= Tdefine[R[la, lb], "ba"];
Tdefine[R4[la], "abcd"];
SP`defineOperator[CovD, "∇", CD];
```

Absorb

```
In[*]:= Absorb::usage
```

```
Out[*]:= Absorb[expr, metric, opts] absorbs the rank-2 tensor 'metric' into 'expr'
to raise or lower indices. Options like IndexQs, HeadQs, and CovDs
control which indices, objects, and derivatives are affected.
```

Absorb 함수는 (계량 텐서일 필요는 없는) rank-2 대칭 텐서를 이용하여 인덱스를 올리거나 내릴 때 사용한다.

첫 번째 인자는 임의의 표현이고, 두 번째 인자는 rank-2 텐서의 이름이다. 옵션으로 IndexQs, HeadQs, CovDs가 있다.

Tdefine[g, "+ba"]

In[*]:= {F[l_a, l_b] × g[ub, uc], RicciCD[l_a, l_b] × g[ua, ub]}
Absorb[%, g]

Out[*]=

$$\{F_{ab} g^{bc}, g^{ab} R_{ab}\}$$

Out[*]=

$$\{F_a^c, R\}$$

In[*]:= g[ua, ub] × BD[l_b, F[l_a, l_d]]
Absorb[%, g]

Out[*]=

$$\partial_a F_{bd} g^{ba}$$

Out[*]=

$$\partial^a F_{ad}$$

공변 연산자가 있는 경우:

In[*]:= g[ub, ud] × CD[l_a, F[l_b, l_c]]
Absorb[%, g]

Out[*]=

$$\nabla_a F_{bc} g^{bd}$$

Out[*]=

$$\nabla_a F^d_c$$

BD와 LD는 공변 연산자가 아니다:

In[*]:= g[ub, uc] × CD[l_e, BD[l_f, R4[l_a, l_b, l_c, l_d]]]
Absorb[%, g]

Out[*]=

$$\nabla_e \partial_f R4_{abcd} g^{bc}$$

Out[*]=

$$\nabla_e \partial_f R4_{abcd} g^{bc}$$

In[*]:= g[ub, uc] × CD[l_e, LD[v, R4[l_a, l_b, l_c, l_d]]]
Absorb[%, g]

Out[*]=

$$\nabla_e \mathcal{L}_v R4_{abcd} g^{bc}$$

Out[*]=

$$\nabla_e \mathcal{L}_v R4_{abcd} g^{bc}$$

계량 텐서가 여러 개 있는 경우:

```
In[*]:= R4[la, lb, lc, ld] × g[ua, ue] × g[ub, uf]
Absorb[%, g]

Out[*]=

$$g^{ae} g^{bf} R_{abcd}$$


Out[*]=

$$R^{ef}_{cd}$$


In[*]:= (R[la, lb] + F[la, lb]) g[ua, ub]
Absorb[%, g]

Out[*]=

$$F_{ab} g^{ab} + g^{ab} R_{ab}$$


Out[*]=

$$F_a^a + R_a^a$$

```

IndexedObject의 Log와 Power는 ScalarFunction이다:

```
In[*]:= {Log[f[] × R[la, lb] × g[ua, ub]],
R[la, lb] + g[la, lb] Tscalar[R[lc, ld] × g[uc, ud]]^n}
Absorb[%, g]

Out[*]=

$$\{ \text{Log}[f g^{ab} R_{ab}], R_{ab} + g_{ab} (g^{cd} R_{cd})^n \}$$


Out[*]=

$$\{ \text{Log}[f R_a^a], R_{ab} + g_{ab} (R_c^c)^n \}$$

```

Option을 이용하여 임의의 연산자를 (임시로) 공변 도함수로 설정할 수 있다:

```
In[*]:= g[ub, ud] × BD[la, F[lb, lc]]
Absorb[%, g, CovDs → {BD}]

Out[*]=

$$\partial_a F_{bc} g^{bd}$$


Out[*]=

$$\partial_a F^d_c$$

```

IndexQs에 대한 옵션이 없으면 두 번째 인자인 계량 텐서의 Kind와 동일한 인덱스 쌍만 Absorb에서 고려한다:

```
In[*]:= SetDefaultKind[Capital]
KindOf /@ {CD, Metricg}

Out[*]=
{Capital, Capital}
```

In[*]:= {F[lA, lB] × g[uB, uC], RicciCD[lA, lB] × g[uA, uB], RicciCD[lA, lB] × g[uA, uB]}
Absorb[%, g]

Out[*]=

$$\{F_{aA} g^{Ac}, g^{ab} R_{ab}, g^{AB} R_{AB}\}$$

Out[*]=

$$\{F_{aA} g^{Ac}, g^{ab} R_{ab}, g^{AB} R_{AB}\}$$

In[*]:= Absorb[RicciCD[lA, lB] × Metricg[uA, uB], Metricg]

Out[*]=

R

F의 Kind는 Latin이기 때문에 Absorb가 동작하지 않는다:

In[*]:= KindOf[F]

Out[*]=

Latin

In[*]:= F[lA, lB] × Metricg[uB, uC]
Absorb[%, Metricg]

Out[*]=

$$F_{AB} g^{BC}$$

Out[*]=

$$F_{AB} g^{BC}$$

In[*]:= %% // SyntaxCheck

General: IA is not a valid index.

Out[*]=

$$g^{BC} F_{AB}$$

RicciCD의 Kind는 DefaultKind이다:

In[*]:= KindOf[RicciCD]

Out[*]=

Capital

In[*]:= {Metricg[uB, uD] × CD[lA, RicciCD[lB, lC]],
Metricg[uB, uD] × CD[lA, F[lB, lC]], Metricg[ub, ud] × CD[lA, F[lb, lc]]}
Absorb[%, Metricg]

Out[*]=

$$\{\nabla_A R_{BC} g^{BD}, \nabla_a F_{AC} g^{AD}, \nabla_a F_{bc} g^{bd}\}$$

Out[*]=

$$\{\nabla_A R^D_c, \nabla_a F_{AC} g^{AD}, \nabla_a F_{bc} g^{bd}\}$$

```
In[*]:= (RicciCD[lA, lb] + F[lA, lb]) Metricg[uA, uB]
Absorb[%, Metricg]
```

```
Out[*]=
```

$$F_{Ab} g^{AB} + g^{AB} R_{Ab}$$

```
Out[*]=
```

$$F_{Ab} g^{AB} + R^B_b$$

사실 RicciCD[lA,lb]는 잘못된 표현이다:

```
In[*]:= RicciCD[lA, lb]
% // SyntaxCheck
```

```
Out[*]=
```

R_{Ab}

General: lb is not a valid index.

```
Out[*]=
```

R_{Ab}

IndexQs에 대한 옵션이 있으면 그것을 만족하는 인덱스 쌍만 Absorb에서 고려한다:

```
In[*]:= (RicciCD[lA, lb] + F[lA, lb]) Metricg[uA, uB]
Absorb[%, Metricg, IndexQs → {KindIndexQ[Latin]}]
```

```
Out[*]=
```

$$F_{Ab} g^{AB} + g^{AB} R_{Ab}$$

```
Out[*]=
```

$$F_{Ab} g^{AB} + g^{AB} R_{Ab}$$

```
In[*]:= SetDefaultKind[Greek];
Tdefine[GaugeF[lμ, lv, la], "-bac", PrintAs → "F"];
DefDerivativeOperator[CovD, "D"];
DefMetric[phi, "φ", Latin]
```

```
In[*]:= expr = {GaugeF[lμ, lv, la] × phi[ua, ub],
  phi[la, lc] × phi[ua, ub], GaugeF[lμ, lv, lA] × phi[uA, ub]}
Absorb[expr, phi]
```

```
Out[*]=
```

$$\{F_{\mu\nu a} \phi^{ab}, \phi_{ac} \phi^{ab}, F_{\mu\nu A} \phi^{Ab}\}$$

```
Out[*]=
```

$$\{F_{\mu\nu}^b, \delta_c^b, F_{\mu\nu A} \phi^{Ab}\}$$

```
In[*]:= Absorb[expr, phi, IndexQs → {KindIndexQ[Latin]}]
```

```
Out[*]=
```

$$\{F_{\mu\nu}^b, \delta_c^b, F_{\mu\nu A} \phi^{Ab}\}$$

```
In[*]:= {Metricg[uρ, uσ] × CD[lμ, RicciCD[lν, lρ]],
  Metricg[uρ, uσ] × CD[lμ, GaugeF[lν, lρ, ua]],
  Metricg[ub, ud] × CD[lμ, F[lb, lc]]}
Absorb[%, Metricg]
```

```
Out[*]= {∇μRνα gασ, ∇μFναa gασ, ∇μFac gad}
```

```
Out[*]= {∇μRνσ, ∇μFνσa, ∇μFac gad}
```

```
In[*]:= expr = phi[ub, ud] × BD[la, phi[lb, lc]]
```

```
Out[*]= ∂aφbc φbd
```

```
In[*]:= Absorb[expr, phi, CovDs → {BD}] (* BD[la, Kdelta[lc, ud]] => 0 *)
```

```
Out[*]= 0
```

기본 설정으로 복귀한다:

```
In[*]:= SetDefaultKind[Latin]
```

Check

두 번째 인자는 rank 2인 대칭 텐서의 이름이어야 한다:

```
In[*]:= Absorb[F[la, lb], R4]
```

Msg: R4 is not a rank -2 tensor. It has rank 4

```
Out[*]= Fab
```

```
In[*]:= GetSymmetry[RicciCD] // GStoString
```

```
Out[*]= +ba
```

```
In[*]:= F[la, lb] × RicciCD[ua, ub]
Absorb[%, RicciCD]
```

```
Out[*]= Fab Rab
```

```
Out[*]= Faa
```

```
In[*]:= UndefTensor[GaugeF];
UndefDerivativeOperator[CovD]
```

Absorbg

$In[*]:=$	Absorbg::usage
$Out[*]:=$	Absorbg[expr, opts] is a shorthand for Absorb[expr, Metricg, opts], using the default metric tensor `Metricg`.
Metricg 텐서를 이용하여 인덱스를 올리거나 내릴 때 사용한다. 인자는 임의의 표현이고, 옵션으로 IndexQs, HeadQs, CovDs가 있다. Absorbg[expr, opts]는 Absorb[expr, Metricg, opts]와 같다. 이 함수는 Metricg가 당연히 존재해야 하므로 MetricgFlag이 On이어야 한다.	
$In[*]:=$	Off[MetricgFlag]
$In[*]:=$	{F[la, lb] × Metricg[ub, uc], RicciCD[la, lb] × Metricg[ua, ub]} Absorbg[%]
$Out[*]:=$	{F _{ab} Metricg[ub, uc], Metricg[ua, ub] R _{ab} }
$Out[*]:=$	Absorb[expr, metric, opts] absorbs the rank-2 tensor 'metric' into 'expr' to raise or lower indices. Options like IndexQs, HeadQs, and CovDs control which indices, objects, and derivatives are affected.
$In[*]:=$	On[MetricgFlag]
$In[*]:=$	{F[la, lb] × Metricg[ub, uc], RicciCD[la, lb] × Metricg[ua, ub]} Absorbg[%]
$Out[*]:=$	{F _{ab} g ^{bc} , g ^{ab} R _{ab} }
$Out[*]:=$	{F _a ^c , R}

PutMetric and PullOutMetric

PutMetric

$In[*]:=$	PutMetric::usage
$Out[*]:=$	PutMetric[expr, index, opts] explicitly raises or lowers the specified 'index' in 'expr' by multiplying with the appropriate metric tensor.

Absorb와 반대로, 계량 텐서를 이용하여 하나의 인덱스를 올리거나 내릴 때 사용한다.

```
In[*]:= Tdefine[T, "*"]; Tdefine[v, 1]
T[ub, lc, ld, lb]
```

```
Out[*]=
```

$$T^a_{cda}$$

위의 출력은 ResetDummies의 자동적 호출에 의해 인덱스 쌍의 기호 (ub,lb)가 (ua,la)로 재조정된 결과이다. 따라서 PutMetric의 두 번째 인자인 내리거나 올릴 인덱스의 이름은 **출력을 마친 이 후**에나 결정됨을 기억하자.

```
In[*]:= expr = %
```

```
Out[*]=
```

$$T^a_{cda}$$

PutMetric의 사용법: 첫 번째 인자는 텐서 표현, 두 번째 인자는 내리거나 올리길 원하는 인덱스이다.

```
In[*]:= PutMetric[expr, ua]
% // Absorbg
```

```
Out[*]=
```

$$g^{ab} T_{bcda}$$

```
Out[*]=
```

$$T^a_{cda}$$

```
In[*]:= PutMetric[expr, la]
% // Absorbg
```

```
Out[*]=
```

$$g_{ab} T^{a\ b}_{cd}$$

```
Out[*]=
```

$$T^a_{cda}$$

연산자가 있는 경우

```
In[*]:= BD[ua, CD[ub, T[la, lb, lc, ud]]]
```

```
Out[*]=
```

$$\partial^a \nabla^b T_{abc}{}^d$$

```
In[*]:= PutMetric[%, la]
PutMetric[%%, ub]
```

```
Out[*]=
```

$$\partial^a g_{ab} \nabla^e T^b{}_{ec}{}^d + \partial^a \nabla^b T^e{}_{bc}{}^d g_{ae}$$

```
Out[*]=
```

$$\partial^a g^{be} \nabla_e T_{abc}{}^d + \partial^a \nabla_b T_{aec}{}^d g^{eb}$$

In[*]:= CD[ua, CD[ub, T[la, lb, lc, ud]]]

Out[*]=

$$\nabla^a \nabla^b T_{abc}^d$$

In[*]:= PutMetric[%, la]

PutMetric[%%, ub]

Out[*]=

$$\nabla^a \nabla^b T_{bc}^e g_{ae}$$

Out[*]=

$$\nabla^a \nabla_b T_{aec}^d g^{eb}$$

In[*]:= v[ld] × LD[v, T[la, ud, le, lf]]

Out[*]=

$$\mathcal{L}_v T_{a\ ef}^b v_b$$

In[*]:= PutMetric[%, ub]

Out[*]=

$$\mathcal{L}_v g^{bc} T_{acef} v_b + \mathcal{L}_v T_{abef} g^{cb} v_c$$

LD type의 연산자가 텐서가 아닌 표현에 작용하면 적절한 텐서 연산이 아니므로 PutMetric은 아무런 효과가 없다.

In[*]:= LD[v, BD[la, T[lb, ud, le, lf]]]

Out[*]=

$$\mathcal{L}_v \partial_a T_b^d{}_{ef}$$

In[*]:= PutMetric[%, lb]

Out[*]=

$$\mathcal{L}_v \partial_a T_b^d{}_{ef}$$

잘못된 입력인 경우에 잘못된 결과를 줄 수 있다:

In[*]:= expr = BD[ua, CD[ub, T[lc, lb, ld, ue]]] × v[lb] (* lb is duplicated *)

Out[*]=

$$\partial^a \nabla^b T_{cbd}^e v_b$$

In[*]:= PutMetric[expr, lb] (* 두 개의 lb 모두 변환됨! *)

Out[*]=

$$\partial^a g_{bf} \nabla^b T_c^f{}^e g_{bg} v^g + \partial^a \nabla^b T_c^f{}^e g_{bf} g_{bg} v^g$$

DefaultKind가 아닐 때의 연산 결과를 보기 위해 Capital Kind의 텐서 표현을 사용하자:

In[*]:= Tdefine[T[LA], "*"]; Tdefine[v[LA], "a"];
DefDerivativeOperator[CCovD, "D", Capital]

Capital Kind에는 아직 계량 텐서가 정의되지 않았다:

In[*]:=

```
v[lA]
PutMetric[%, lA]
```

Out[*]=

 v_A

Out[*]=

 v_A

Capital Kind에 계량 텐서를 도입하여 실행 결과를 확인한다:

In[*]:=

```
DefMetric[Phi, "ϕ", Capital]
```

In[*]:=

```
T[lA, uB]
PutMetric[%, uB]
```

Out[*]=

 T_A^B

Out[*]=

 $\Phi^{BC} T_{AC}$

In[*]:=

```
CCovD[lC, T[lA, uB]]
PutMetric[%, lC]
```

Out[*]=

 $D_C T_A^B$

Out[*]=

 $D^D T_A^B \Phi_{CD}$

기본 설정으로 복귀한다:

In[*]:=

```
UndefMetric[Phi];
UndefDerivativeOperator[CCovD];
Tdefine[T, "*"];
Tdefine[v, 1]
```

PullOutMetric

In[*]:=

```
PullOutMetric::usage
```

Out[*]=

```
PullOutMetric[expr, opts] converts 'up/dn' states of indices in 'expr'
to pre-defined ones, pulling out the corresponding metric tensors.
```

텐서를 정의할 때 설정된 위/아래 첨자의 위치로 (계량 텐서를 사용하여) 되돌린다. (위/아래 첨자를 특별히 정하지 않았으면 기본적으로 아래 첨자로 지정된다.)

텐서 T의 기본 인덱스 위치가 모두 위 첨자가 되도록 정의한 후 실행 결과를 확인한다:

In[*]:= **Tdefine**[T[ua, ub]]

In[*]:= **T**[ua, ub]
% // PullOutMetric

Out[*]=

T^{ab}

Out[*]=

T^{ab}

In[*]:= **T**[la, ub]
% // PullOutMetric
% // Absorbg

Out[*]=

T_a^b

Out[*]=

$g_{ac} T^{cb}$

Out[*]=

T_a^b

In[*]:= **T**[la, lb]
% // PullOutMetric
% // Absorbg

Out[*]=

T_{ab}

Out[*]=

$g_{ac} g_{bd} T^{cd}$

Out[*]=

T_{ab}

CD type 연산자의 첫 번째 인자는 아래 첨자가 기본 위치인 인덱스이다:

In[*]:= **BD**[lc, T[ua, ub]]
% // PullOutMetric

Out[*]=

$\partial_c T^{ab}$

Out[*]=

$\partial_c T^{ab}$

```
In[*]:= BD[uc, T[ua, ub]]
% // PullOutMetric
% // Absorbg
```

```
Out[*]=
```

$$\partial^c T^{ab}$$

```
Out[*]=
```

$$\partial_d T^{ab} g^{cd}$$

```
Out[*]=
```

$$\partial^c T^{ab}$$

```
In[*]:= CD[ud, BD[uc, T[ua, ub]]]
% // PullOutMetric
% // Absorbg
```

```
Out[*]=
```

$$\nabla^d \partial^c T^{ab}$$

```
Out[*]=
```

$$\nabla_e \partial_f T^{ab} g^{cf} g^{de}$$

```
Out[*]=
```

$$\nabla^d \partial^c T^{ab}$$

DualStar

참고: R. M. Wald, Chap.4 Problems 2-(a)

$$*(\alpha_{a_1 \dots a_p}) = (*\alpha)^{b_1 \dots b_{n-p}} = \frac{1}{p!} \alpha_{a_1 \dots a_p} \epsilon^{a_1 \dots a_p b_1 \dots b_{n-p}}$$

임의의 차원에서 DualStar 연산을 하도록 준비한다:

```
In[*]:= GetDimension[Latin] = n;
GetDimension[Latin]
```

```
Out[*]=
```

n

임의의 rank를 갖는 완전 반대칭 텐서를 정의한다:

```
In[*]:= Tdefine[A, "*-"]
```

DualStar의 두 번째 인자는 Epsilon 텐서의 인덱스 중에서 Free 인덱스에 해당한다. 그 인덱스의 적절함은 사용자의 몫이다:

```
In[*]:= DualStar[A[ua, lb], {lc, ld}]
```

```
Out[*]=
```

$$\frac{1}{2} A^a_b \epsilon_a^b{}_{cd}$$

In[*]:= DualStar[A[ua, lb, ud], {lc, le}]

Out[*]=

$$\frac{1}{6} A^a_d \in_a^b dce$$

In[*]:= DualStar[A[ua, lb, ud, uc], {le}]

Out[*]=

$$\frac{1}{24} A^a_{cd} \in_a^b cde$$

In[*]:= DualStar[A[ua, lb, ud, uc]]

Out[*]=

$$\frac{1}{24} A^a_{cd} \in_a^b cd$$

In[*]:= DualStar[A[ua, lb], {lc}]

Out[*]=

$$\frac{1}{2} A^a_b \in_a^b c$$

Epsilon 텐서에 대한 인덱스 개수의 적절함을 자동적으로 확인하기 위해 공간 차원의 값을 입력하자:

In[*]:= SetDimension[4]

In[*]:= DualStar[A[ua, lb], {lc}]

Msg: Invalid numbers of indices: {lc}

Out[*]=

\$Failed

In[*]:= DualStar[A[ua, lb], {lc, ld}]

Out[*]=

$$\frac{1}{2} A^a_b \in_a^b cd$$

원래의 설정으로 복귀한다:

In[*]:= ClearDimension[]

Capital 계열에서 DualStar 동작을 확인하자:

In[*]:= Tdefine[CA, "*", Capital]

In[*]:= DualStar[CA[uA], {lB}, Capital]

Out[*]=

$$CA^A \in [Capital]_{AB}$$

In[*]:= DualStar[CA[uA, lB], {lC}, Capital]

Out[*]=

$$\frac{1}{2} CA^A_B \in [\text{Capital}]^B_{A\ C}$$

In[*]:= DualStar[CA[uA, lD, lB], {lC}, Capital]

Out[*]=

$$\frac{1}{6} CA^A_{BD} \in [\text{Capital}]^{BD}_{A\ C}$$

Check

In[*]:= DualStar[CA[uA, uB]]

Out[*]=
 ... Msg: Invalid numbers of indices: {}

\$Failed

In[*]:= DualStar[A[ua, ub], {ua}]

Out[*]=
 ... Msg: ill-formed indices: {ua}

\$Failed

In[*]:= SetDimension[3]

In[*]:= DualStar[A[ua, lb, uc, ud]]

Out[*]=
 ... Msg: Invalid numbers of free indices: {ua, lb, uc, ud }

\$Failed

In[*]:= DualStar[A[ua, ub], {lc, ld}]

Out[*]=
 ... Msg: Invalid numbers of indices: {lc, ld }

\$Failed

In[*]:= ClearDimension[]